

Research Statement

Morteza Zakeri, PhD in Computer Engineering (Software)

August 2026

Towards effective, efficient, and economic automation in the software realm.

"Fools ignore complexity. Pragmatists suffer it. Some can avoid it. Geniuses 'remove' it."

✠ Alan Perlis (1922—1990)

1 Overview

My area of interest is Automated and Intelligent Software and Security Engineering (AISSE) to make software development and maintenance effective, efficient, and economical. My research has focused on developing intelligent systems and tools for software quality measurement and improvement, test suite generation and minimization, technical debt estimation and reduction, code recommendation, and software engineering methodologies¹. The main systems I have developed include (i) a testability prediction system [1]–[4] that uses machine learning models to predict the testability of the classes and systems under test; (ii) an multi-dimensional automated refactoring engine [5]–[8] that automates the application of 18 refactoring operations at the source code level to improve eight software quality attribute recently has formed QualCode [9] a tool that realize ISO/IEC 25010:2011; and (iii) a format-aware fuzzer [10] that generates test data for software applications with complex input formats, such as PDF readers and a domain model test data generator [11].

The underlying techniques in all proposed systems are white-box compilers (compiler II), model-driven program transformation, data-driven algorithms (software II), and evolutionary computing. I have published some systematic literature reviews on the above topics [12]–[15]. They mostly help me propose new software development methodologies and techniques rooted in requirements engineering [16]–[18]. The proposed tools and systems significantly advance the state of the art in Automated Software Engineering (ASE) and enable new paradigms for improving the quality of real-world software systems.

During past years, I have extended my research beyond software engineering to develop intelligent systems for other engineering domains, particularly material engineering [19]–[22], civil engineering [23], and biomedical engineering [24], [25]. Bringing intelligence to materials engineering, I develop surrogate machine learning models to predict the properties of complex materials, including alloys and glasses, and couple these models with evolutionary optimization to guide inverse design and discovery of new materials.

Overall, my main stream research indicates that (i) downstream tasks in automated source code engineering can be unified and formulated as a general problem of source code similarity measurement with simple models [12]; (ii) the performance and correctness of data-driven source code engineering techniques, such as code smell detection, highly depends on datasets that currently contain many incorrect samples [13]; and (iii) shallow learning in some code-related tasks, such as method name recommendation, is more effective and efficient than deep learning [26]. The rest of this research statement discusses my research motivation and goals, summarizes the previous and present state of my research during my M.Sc., Ph.D.², and post doctorate and outlines the future directions of the research that I intend to pursue.

¹<https://www.m-zakeri.github.io/lab/en/>

²<https://m-zakeri.github.io/PhD/>

2 Motivation, importance, and goals of my research

Software engineering (SE) is intricate because software systems and ecosystems are inherently complex, intangible, and unpredictable. Software engineers must deal with non-trivial activities such as testing, quality assurance, and maintenance during the software development life cycle (SDLC). Efficient automation of these activities not only increases the process quality and product reliability but also leads to economic savings. Unfortunately, in practice, search-based software engineering (SBSE) techniques fail to automate emerging tasks, such as measuring and improving software quality, test data generation for complex format inputs, and code recommendation. I use machine learning software engineering (MLSE) alongside SBSE to address open problems in automating SDLC activities, including quality prediction, code suggestion, refactoring, testing, debugging, repair, and maintenance [1]–[11].

Automation approaches must deal with a variety of software development artifacts, *e.g.*, requirements, production codes, test codes, and design diagrams, requiring different preprocessing and artifact representation mechanisms. However, most research works have been dedicated to automating code-related tasks [12]. I have found that the automation of the majority of these activities can be achieved through the effective representation of program source codes and, in general, a proper artifact similarity measurement instrument [12]–[15]. It is highly important to uncover such common aspects by interpreting newly developed approaches to improve the generalization and applicability of proposed solutions and provide a solid theoretical framework for ASE.

My research aims to provide intelligent and off-the-shelf methods and tools for software engineers to build their software systems with the highest quality, lowest cost, and minimum time. These tools are expected to be executed on developers' computer systems with low memory, latency, and overhead. My long-term goal is to generalize the proposed automated SE tools to work on different programming languages, various software projects' instances, and real-world and large-scale software applications, with good accuracy and performance.

During my graduate course, I had the chance to collaborate with researchers worldwide who are pioneers in the field and have similar research interests, in addition to my supervisor¹ and advisor², to realize some aforementioned goals. I collaborated with Dr. Fabio Palmoba (University of Salerno)³, Dr. Mohamed Wiem Mkaouer (Rochester Institute of Technology and University of Michigan-Flint)⁴, Prof. Chanchal K. Roy (University of Saskatchewan)⁵, Prof. Alexander Chatzigeorgiou (University of Macedonia)⁶, and Prof. Burak Turhan (University of Oulu)⁷ research laboratories. I am highly motivated to pursue and extend such collaborations to advance the AISE field and expand its benefits to the community.

My long-term research vision is to develop intelligent engineering systems that combine machine learning, program analysis, evolutionary computation, and domain knowledge to automate complex design, development, and maintenance activities. My primary research domain is automated and intelligent software engineering (AISE), where I investigate intelligent methods for software quality assurance, testing, program transformation, and software development [1]–[11]. In parallel, I have extended these computational methodologies to materials science and other engineering domains, including materials property prediction, explainable modeling, and inverse design [19]–[22]. Across these domains, my research follows a common principle: learning reliable and interpretable surrogate models from complex engineering data and coupling them with efficient search and optimization methods to recommend high-quality solutions. This perspective enables my work to contribute both to fundamental AI and intelligent-systems research and to interdisciplinary engineering applications.

¹<https://www.researchgate.net/profile/Saeed-Parsa>

²https://webpages.iust.ac.ir/m_ashtiani/

³<https://fpalomba.github.io/>

⁴<https://mkaouer.net/>

⁵<https://clones.usask.ca/>

⁶https://sites.uom.gr/achat/public_html/

⁷<https://turhanb.net/>

3 Research in automated and intelligent software engineering

During the course of my M.Sc., I applied deep learning techniques to learn the structure of complex file format grammar and build a generative model to generate new test data, improving the effectiveness of file format fuzzers [10]. Regardless of inventing sophisticated test data generation methods, it was found that the test coverage is still very low in some of the software under test (SUT) due to software testability issues. Indeed, the low testability of the SUT leads to increasing test effort while decreasing test coverage. Therefore, in my Ph.D., I decided to focus on measuring and improving the testability of software systems artifacts to prepare the artifacts for testing before using automatic testing tools on a given software [2], [3], [27]. The following summarizes the technical aspects of my research work during my M.Sc. and Ph.D. courses.

Deep learning-based test data generation with DeepFuzz Generating test data for complex input files, such as PDFs, with the ability to achieve high code coverage and find more faults requires that test data pass all of the parsing stages of the SUT. Randomly generating such test data leads to very low code coverage, which cannot guarantee the absence of bugs and the reliability of software. On the other hand, generating test data from grammar requires extracting the grammar or model of the file manually, which is expensive and time-consuming. I proposed DeepFuzz [10], a fuzzer that utilizes a deep neural language model to learn the structure of complex input formats. The learned model is then sampled with various strategies to generate new fuzzed files as test data. The first problem we encountered was finding a mechanism to distinguish between data and metadata. To do so, we applied a reasonable trick: As metadata repeated in almost every sample of a file format, the learned model predicts the metadata with higher probability in comparison to the data. By putting a threshold on the model output, the data and metadata are distinguishable.

Aims to seek faults in software by fuzzing techniques; the second problem was how to determine which byte should be fuzzed to reveal failures in the SUT. This can be done by targeting different stages of input processing that the SUT uses. For example, if we would like to fuzz the parser, we should fuzz metadata because the parser usually deals with metadata to validate the format of the input file and extract data. On the other hand, if we would like to fuzz the execution stage, it may be better to fuzz data. The ability of the learned model to distinguish data and metadata is used to determine the place of fuzz in the file. In addition to determining the place of the fuzz, we should inform the value for replacing the third problem we addressed. As we know, the goal of fuzzing is creating the malformed input, and hence the most inappropriate byte is expected to be replaced in the input file. Again, the most inappropriate byte can be determined by the learned neural language model. It is enough to select a byte with the lowest likelihood instead of the highest likelihood used in the default manner.

Contributions. DeepFuzz [10] significantly outperforms previous file format fuzzer, such as AFL and Augmented-AFL, in terms of the achieved code coverage level for the SUT with complex input formats, specifically, PDF readers.

Learning to predict test effectiveness with Coverageability Predicting test effectiveness is vital since finding all faults in the SUT is not a decidable problem. Test effectiveness depends on the extent to which the selected test cases cover the SUT, which is only computable after testing the SUT. For this reason, measuring test coverage is expensive and time-consuming, requiring executable production code, installed dependencies, available resources, etc. I introduced a new quality attribute, Coverageability, to describe the test effectiveness based on test coverage and a machine-learning regression model to estimate the Coverageability of each unit in the program under test [2]. Coverageability is the extent to which a given unit under test, *e.g.*, a class, is expected to be covered by a test case.

There are three main challenges while adopting machine learning to build a Coverageability prediction model: (i) selecting metrics to build the feature space. Initially, 54 known software metrics were collected. The feature space was extended by adding over 200 statistical and lexical metrics to obtain better performance; (ii) providing an adequate number of samples to train and test the machine learning model. We computed the actual

value of Coverageability using the size and the geometric average of statement, branch, and mutation coverage of the test suite generated by running EvoSuite on 23000 Java classes; (iii) selecting the most appropriate combination of test adequacy criteria, representing test effectiveness. Our experiments show that combining coverage criteria provides a relatively more accurate indication of test effectiveness than using only a single criterion. In summary, Coverageability as a new quality attribute can best be used to estimate the test effectiveness of SUT just in time, and developers can continuously change their code in such a way that achieves the maximum Coverageability.

Contributions. The proposed Coverageability prediction models [2] have improved the MAE by 5.78% and R^2 score by 20.71% compared with the state-of-the-art approach for test effectiveness measurement, which only relies on branch coverage.

Software testability prediction with ADAFEST Despite a large number of studies on software testability, we observed that the relationship between testability and test adequacy criteria had not been explored empirically, and testability metrics are still far from measuring the actual testing effort. I combined Coverageability with test effort (*i.e.*, time budget and size of the test suite) to formalize a mathematical model for measuring software testability based on the ISO/IEC 25010:2011 standard [3]. Again, a machine learning model was trained on a large corpus of already tested software projects to build a testability prediction model for just-in-time evaluation of the SUT testability. Connecting runtime information to the static properties of the program throughout the learning-based algorithms is a key point in measuring software quality, specifically testability. Using machine learning interpretation techniques, mainly permutation importance analysis, my research automatically designated the most influential source code metrics affecting the SUT testability.

The proposed ADAFEST¹ [3], [27] tool can be used to predict the testability of SUT without any need to program execution. Our experiments revealed that applying automated refactoring operations significantly improves SUT testability, and the testability prediction model guides developers to enhance the testability of their classes while coding. Previous studies on improving software testability with automated refactoring have yielded inconclusive results. Investigating the impact of automated refactoring on testability is crucial since refactoring is a kind of program transformation that, unlike testability transformation techniques, preserves behavior and produces a permanent and usable version of the software. Moreover, testing is a non-trivial and resource-consuming software engineering activity that entirely depends on system testability. It is possible to extend the proposed testability prediction model to work at the design level, where not all source code metrics are available, and other artifacts such as class and package diagrams must be targeted for the testability measurement and improvement.

Contributions. AFASET [3] is the first tool that measures testability based on the ISO/IEC 25010:2011 standard with the help of intelligent models. It enables the automatic determination of important factors affecting software testability in practice.

Flipped boosting of automatic test data generation tools with CodART Soon after I developed the first testability prediction model and found the important metrics affecting testability, I realized that it is possible to improve the performance of automatic test data generation tools, such as DeepFuzz [10] and EvoSuite, without directly changing their internal structure and algorithms. The idea was to maximize the testability of a SUT by finding the most appropriate refactoring sequence and applying that refactoring sequence to the SUT before testing it using test data generation tools. Testability prediction mode can be used as a fitness function in search-based software refactoring methods to find the refactoring sequences maximizing testability. This way, the performance of test data generation tools regarding test adequacy criteria is improved since the algorithms used in these tools already work fine on SUT with an acceptable degree of testability. The main challenge was that despite many studies on search-based software refactoring, there were only a few tools that operate at the source code level. Most search-based refactoring studies use a simplified intermediate model

¹A data-driven apparatus for estimating software testability (<https://doi.org/10.24433/C0.9048896.v1>)

extracted from the source code to work on. Moreover, the implementation of many automated refactoring papers is not available, making their usage questionable and complicated.

To address the above problems, I designed and developed CodART [5], [8], an automated refactoring toolkit that is capable of identifying and applying 18 refactoring operations at the source code level. The proposed refactoring engine can be used to refactor Java programs. However, it is modular, well-documented, and provides an API to add support for other programming languages and refactoring. CodART uses ANTLR¹, which generates the program parse tree and provides the depth-first search (DFS) algorithm to visit parse tree nodes, along with a call-back mechanism that calls a specific method when the visiting algorithm enters or exits the node. Using this mechanism, CodART finds the locations that should be modified for a given refactoring operation. In addition, call-back methods are attached to grammar rules to improve performance. I recently published the underlying program analysis engine (OpenUnderstand) on PyPi².

CodART improves the testability of Java projects by an average of 29.95%, provided that a relevant and precise testability prediction objective is used during refactoring sequence optimization. As a result, the performance of automated test data generation tools such as EvoSuite increased by 16.08% and 15.24% for branch and statement coverage. I called this approach *flipped or reverse boosting of automatic test data generation tools*, in which the performance of the automatic test data generator improved without directly modifying the tool. CodART also takes care of seven other quality attributes, including modularity and QMOOD attributes, due to using the NSGA-III many-objective optimization algorithm. CodART has demonstrated a quality gain of 20.06%. Moreover, the manual precision of the suggested refactoring solutions has improved by 19.5% compared to state-of-the-art refactoring tools. The proposed flipped boosting mechanism can be generalized to use in other areas of ASE, such as automatic software fault localization. The thinking paradigm behind the flipped boosting mechanism is appreciated in a way that, instead of trying to build a system for solving a complex problem, tries to make a simplified version of the problem such that it can be solved with existing solvers.

Contributions. CodART [5], [8] is the first automatic refactoring engine that considers testability improvement while caring about human-centric software quality attributes, such as reusability and understandability.

Software entity name suggestion apparatus (SENSA) One of the main prerequisites of successful program transformation with refactoring from a development viewpoint is to have a transformed program with the appropriate names for newly created entities, mainly classes and methods. Refactoring operations such as extract classes and extract methods create new classes and methods which require a proper name. State-of-the-art naming techniques use deep learning to compute the methods' similarity considering their textual contents. They highly depend on identifiers' names and do not compute semantical interrelations among methods' instructions. Source code metrics compute such semantical interrelations. I have designed and developed SENSA [26], [28], a tool that uses using source code metrics and a K-nearest neighbor learning algorithm to measure semantical and structural cross-project similarities for recommending method names. SENSA achieves a precision of 65.49% with a recall of 59.15% on 420528 Java methods, which is the highest score among state-of-the-art approaches in this field. In addition, the average response time of method recommendation is 0.52 seconds, which is lower than deep learning-based approaches. Finally, SENSA applies to used for programming languages other than Java, mainly C# and C++. It achieves an F1 score of 35.91% on predicting C++ method names with no change in the dataset and methodology, indicating its language neutrality. Overall, my research shows that a shallow learning approach based on source code metrics achieves greater efficiency and effectiveness in method name recommendations than deep learning approaches.

Contributions. SENSA [26] is a lightweight software entity name recommendation tool that significantly outperforms state-of-the-art deep-learning-based tools in terms of efficiency and effectiveness.

Research in progress My current research is continuing towards building intelligent testability and security assurance systems. It includes, but is not limited to, security analysis and detection of vulnerable and malicious

¹<https://www.antlr.org/>

²<https://pypi.org/project/openunderstand/>

auto generated codes, AI models [29], and complex structured files, mainly PDF and Microsoft Office files [30]. The rapid emergence of large language models, foundation models, and AI-assisted software development is making software quality and security assurance increasingly important. Hence, I am mainly considering the security and quality aspects of this 'wild' automation in my ongoing research.

4 Future directions of my research

My research plans for future work center around my long-term career goal to create generalized intelligent systems for ASE. I briefly outline some important directions for my future research in the area of the AISSE field. There are vast opportunities regarding the topics discussed in my previous and current research, most importantly those concerning downstream tasks on source code, automated refactoring performance, reverse boosting frameworks, and security assurance. I think that what I propose in the following highly helps bridge the gap between researchers and practitioners in the field.

A solid ASE framework based on code similarity and foundation models My recent systematic literature review on source code similarity measurement and clone detection [12] suggests that code similarity can serve as a fundamental computational primitive for a broad range of data-driven software engineering tasks. The same underlying notion of similarity can support clone detection, code recommendation, defect prediction, code-smell detection, vulnerability identification, refactoring recommendation, and software quality measurement. Rather than developing these capabilities as isolated tools, I aim to investigate how they can be unified into a general framework for intelligent and automated software engineering. Advances in foundation models and agentic AI provide an opportunity to extend this vision beyond individual code-generation or recommendation tasks toward autonomous software engineering agents. Such agents can decompose high-level engineering goals, interact with development tools, execute and test code, analyze feedback, and iteratively modify software. A powerful code recommendation tool can be designed and developed to make a super agile and automated software development methodology. A central research challenge is to make these agents reliable, efficient, and trustworthy. LLM-based agents can generate plausible but incorrect solutions, lose contextual information during long-horizon tasks, and make unsafe decisions when interacting with large software environments. I therefore plan to investigate how lightweight and interpretable representations, such as code metrics and structural similarity, can complement foundation models and reduce their computational and reasoning limitations. Achieving this final goal requires attention to some remaining uncovered aspects in source code engineering, described in the following.

Combining deep and shallow learning for code-related tasks Data-driven approaches for automating software engineering tasks require appropriate artifact representation, *e.g.*, source code, design, and requirements representations to extract features and be fed into statistical and learning algorithms. The benefit of deep learning methods on feature extraction from source code is obvious. They remove the manual feature engineering and provide dense and semantical source code representations. Pretrained (large) language models such as BERT and GPT are appropriate systems for automating multiple code-related tasks. However, they require many computational resources, large and high-quality datasets, and fine-tuning effort to produce sound full results. Moreover, deep models are difficult to interpret and explain. Metrics-based source code representation, on the other hand, provides a lightweight, interpretable, language-independent, and efficient method to represent source code as a fixed-length vector. I plan to explore the combination of manual and automatic source code feature engineering methods and their impact on my proposed tools. There are several ways to combine two vectors, including concatenation, linear combination, merging, etc. Therefore, investigating the best combination methods is an open problem.

Connecting code smell detection and automated refactoring Despite significant advances in automated code smell detection and search-based refactoring, the two research areas remain largely disconnected.

Most search-based refactoring approaches initialize the search process randomly, leading to inefficient exploration of the search space and suboptimal solutions. I believe that reliable identification of code smells and refactoring opportunities should guide the search process to improve both efficiency and effectiveness. My recent systematic literature review on code smell datasets [13] further revealed substantial gaps in publicly available datasets and automated support for many code smells and Fowler and Beck's refactoring activities. I plan to develop comprehensive code smell datasets and intelligent refactoring-opportunity detection techniques that can be seamlessly integrated with search-based refactoring frameworks, enabling more effective and practical automated software maintenance.

Automated algorithm design and selection for SE tasks With the increasing number of proposed algorithms and methods for difficult software engineering tasks such as test data generation, fault localization, and search algorithms, selecting the best algorithm for a particular program instance is vital. A common approach to tackling the algorithm-selection problem is to treat it as a classification problem where each instance is described in terms of a vector of hand-designed features, and an instance's class indicates the best-performing algorithm. Algorithm selection has successfully been applied in many domains such as artificial intelligence and combinatorial problems. However, it has rarely been used for difficult SE tasks where there is no dominant algorithm for all problem instances. The main challenges in this area are datasets of problem instances and solutions regarding the SE tasks and artifact representation for training appropriate algorithm selection models. I plan to create datasets and methods for automated algorithm selection in SE tasks such as test data generation. It is another way to amplify the performance of test data generators by running them only on SUTs that are suitable for the test data generation algorithm.

Intelligent security testing for AI-Assisted software The rapid adoption of generative AI and LLM-assisted programming introduces new challenges for software security, including vulnerabilities in AI-generated code, malicious code generation, and the validation of AI-generated software artifacts. However, LLMs alone are often unreliable for these security-critical tasks. I therefore plan to develop hybrid intelligent security-testing approaches that combine LLMs with program analysis, fuzzing, dynamic execution, and evolutionary search. The goal is to use LLMs for generating security hypotheses and candidate tests while relying on rigorous automated analysis and execution to validate them. This framework will target automated vulnerability detection, exploit generation and validation, malicious software detection, and continuous security testing of AI-assisted software systems.

References

- [1] M. Zakeri-Nasrabadi and S. Parsa, "Learning to predict software testability," in *2021 26th International Computer Conference, Computer Society of Iran (CSICC)*, IEEE, Mar. 2021, pp. 1–5, ISBN: 978-1-6654-1241-4. DOI: [10.1109/CSICC52343.2021.9420548](https://doi.org/10.1109/CSICC52343.2021.9420548). [Online]. Available: <https://ieeexplore.ieee.org/document/9420548/>.
- [2] M. Zakeri-Nasrabadi and S. Parsa, "Learning to predict test effectiveness," *International Journal of Intelligent Systems*, Oct. 2021, ISSN: 0884-8173. DOI: [10.1002/int.22722](https://doi.org/10.1002/int.22722). [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1002/int.22722>.
- [3] M. Zakeri-Nasrabadi and S. Parsa, "An ensemble meta-estimator to predict source code testability," *Applied Soft Computing*, vol. 129, p. 109562, Nov. 2022, ISSN: 15684946. DOI: [10.1016/j.asoc.2022.109562](https://doi.org/10.1016/j.asoc.2022.109562).
- [4] M. Zakeri-Nasrabadi, S. Parsa, and S. Jafari, "Measuring and improving software testability at the design level," *Information and Software Technology*, p. 107511, Oct. 2024, ISSN: 09505849. DOI: [10.1016/j.infsof.2024.107511](https://doi.org/10.1016/j.infsof.2024.107511). [Online]. Available: <https://doi.org/10.1016/j.infsof.2024.107511>.

- [5] M. Zakeri-Nasrabadi and S. Parsa, *CodART: Automated source code refactoring toolkit*, version 1.0.0, Jan. [Accessed: 2023-01-18]. DOI: <https://doi.org/10.5281/zenodo.6508415>. (visited on 01/18/2023).
- [6] M. Shahidi, M. Ashtiani, and M. Zakeri-Nasrabadi, "An automated extract method refactoring approach to correct the long method code smell," *Journal of Systems and Software*, vol. 187, p. 111 221, 2022, ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2022.111221>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121222000048>.
- [7] A. Ardalani, S. Parsa, M. Zakeri-Nasrabadi, and A. Chatzigeorgiou, "Supporting single responsibility through automated extract method refactoring," *Empirical Software Engineering*, vol. 29, p. 28, 1 Feb. 2024, ISSN: 1382-3256. DOI: [10.1007/s10664-023-10427-3](https://doi.org/10.1007/s10664-023-10427-3).
- [8] M. Zakeri, F. Abdi, and F. Bagheri, "Enhancing software quality attributes through multi-dimensional refactoring at source-level," *Science of Computer Programming*, vol. 251, p. 103 434, 2026, ISSN: 0167-6423. DOI: <https://doi.org/10.1016/j.scico.2025.103434>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167642325001728>.
- [9] E. Azhir, M. Zakeri, Y. Abedini, and M. Mostafavi Ghahfarokhi, "Qualcode: A data-driven framework for predicting software maintainability based on iso/iec 25010," *Science of Computer Programming*, vol. 250, p. 103 399, 2026, ISSN: 0167-6423. DOI: <https://doi.org/10.1016/j.scico.2025.103399>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167642325001388>.
- [10] M. Zakeri-Nasrabadi, S. Parsa, and A. Kalaei, "Format-aware learn&fuzz: Deep test data generation for efficient fuzzing," *Neural Computing and Applications*, vol. 33, 5 2021, ISSN: 14333058. DOI: [10.1007/s00521-020-05039-7](https://doi.org/10.1007/s00521-020-05039-7).
- [11] R. Golmohammadi, S. Parsa, and M. Zakeri-Nasrabadi, "Dynamic domain testing with multi-agent markov chain monte carlo method," *Soft Computing*, 2024. DOI: [10.1007/s00500-024-09680-5](https://doi.org/10.1007/s00500-024-09680-5).
- [12] M. Zakeri-Nasrabadi, S. Parsa, M. Ramezani, C. Roy, and M. Ekhtiarzadeh, "A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges," *Journal of Systems and Software*, p. 111 796, 2023, ISSN: 0164-1212. DOI: <https://doi.org/10.1016/j.jss.2023.111796>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0164121223001917>.
- [13] M. Zakeri-Nasrabadi, S. Parsa, E. Esmaili, and F. Palomba, "A systematic literature review on the code smells datasets and validation mechanisms," *ACM Computing Surveys*, May 2023, ISSN: 0360-0300. DOI: [10.1145/3596908](https://doi.org/10.1145/3596908).
- [14] F. Bagheri-Galle, S. Parsa, and M. Zakeri, "A systematic literature review on transformation for testability techniques in software systems," *Information and Software Technology*, p. 107 788, 2025. DOI: [10.1016/j.infsof.2025.107788](https://doi.org/10.1016/j.infsof.2025.107788).
- [15] N. Abbasi, S. Parsa, and M. Zakeri, "A systematic literature review on design for testability approaches in software systems," *ACM Comput. Surv.*, vol. 58, no. 12, May 2026, ISSN: 0360-0300. DOI: [10.1145/3787493](https://doi.org/10.1145/3787493). [Online]. Available: <https://doi.org/10.1145/3787493>.
- [16] S. Parsa, M. Zakeri-Nasrabadi, and B. Turhan, "Testability-driven development: An improvement to the tdd efficiency," *Computer Standards & Interfaces*, vol. 91, p. 103 877, Jan. 2025, ISSN: 09205489. DOI: [10.1016/j.csi.2024.103877](https://doi.org/10.1016/j.csi.2024.103877). [Online]. Available: <https://doi.org/10.1016/j.csi.2024.103877>.
- [17] M. Zakeri-Nasrabadi and S. Parsa, "Natural language requirements testability measurement based on requirement smells," *Neural Computing and Applications*, Apr. 2024, ISSN: 0941-0643. DOI: [10.1007/s00521-024-09730-x](https://doi.org/10.1007/s00521-024-09730-x). [Online]. Available: <https://link.springer.com/article/10.1007/s00521-024-09730-x>.
- [18] A. Majidzadeh, M. Ashtiani, and M. Zakeri-Nasrabadi, "Multi-type requirements traceability prediction by code data augmentation and fine-tuning ms-codebert," *Computer Standards & Interfaces*, vol. 90, p. 103 850, Aug. 2024, ISSN: 09205489. DOI: [10.1016/j.csi.2024.103850](https://doi.org/10.1016/j.csi.2024.103850).

- [19] A. Khanlari, A. R. Eivani, M. Zakeri, J. Zhou, H. R. Jafarian, and M. Tayebi, "Machine learning-assisted prediction of the hardness of additively manufactured and heat-treated ti-6al-4v alloy," *Results in Engineering*, vol. 30, p. 110727, 2026, ISSN: 2590-1230. DOI: <https://doi.org/10.1016/j.rineng.2026.110727>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2590123026017603>.
- [20] A. Ghasemi, A. R. Eivani, O. Banapour Ghaffari, *et al.*, "Machine learning for predicting high-temperature strength of austenitic stainless steels based on composition and deformation parameters," *Journal of Materials Science*, vol. 60, no. 47, pp. 24665–24683, 2025. DOI: <https://doi.org/10.1007/s10853-025-11805-6>.
- [21] O. Banapour Ghaffari, B. Eftekhari Yekta, and M. Zakeri-Nasrabadi, "Designing high-performance ion-exchangeable glasses with multi-objective optimization and machine learning," *Ceramics International*, Aug. 2024, ISSN: 0272-8842. DOI: <https://doi.org/10.1016/j.ceramint.2024.08.141>. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0272884224035971>.
- [22] O. B. Ghaffari, B. E. Yekta, and M. Zakeri-Nasrabadi, "Estimating "depth of layer" (dol) in ion-exchanged glasses using explainable machine learning," *Materialia*, p. 102027, Jan. 2024, ISSN: 25891529. DOI: [10.1016/j.mtla.2024.102027](https://doi.org/10.1016/j.mtla.2024.102027). [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S2589152924000243>.
- [23] M. Mir, F. Nasirzadeh, M. Zakeri, A. T. Hill, and C. Karmakar, "Assessing neural markers of attention during exposure to construction noise using machine learning classification of electroencephalogram data," *Building and Environment*, vol. 261, p. 111754, Aug. 2024, ISSN: 03601323. DOI: [10.1016/j.buildenv.2024.111754](https://doi.org/10.1016/j.buildenv.2024.111754). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0360132324005961>.
- [24] M. Zakeri-Nasrabadi, H. Tabibi, M. Salmani, M. Torkashvand, and E. Zarepour, "A comprehensive survey on non-invasive wearable bladder volume monitoring systems," *Medical and Biological Engineering and Computing*, vol. 59, 7-8 2021, ISSN: 17410444. DOI: [10.1007/s11517-021-02395-x](https://doi.org/10.1007/s11517-021-02395-x).
- [25] E. Zarepour, M. R. Mohammadi, M. Zakeri-Nasrabadi, *et al.*, "Bilibin: An intelligent mobile phone-based platform to monitor newborn jaundice," *IRANIAN JOURNAL OF ELECTRICAL AND ELECTRONIC ENGINEERING*, vol. 20, no. 3, Sep. 2024. DOI: [10.22068/IJEEE.20.3.3374](https://doi.org/10.22068/IJEEE.20.3.3374). eprint: <http://ijeee.iust.ac.ir/article-1-3374-en.pdf>. [Online]. Available: <http://ijeee.iust.ac.ir/article-1-3374-en.html>.
- [26] S. Parsa, M. Zakeri-Nasrabadi, M. Ekhtiarzadeh, and M. Ramezani, "Method name recommendation based on source code metrics," *Journal of Computer Languages*, vol. 74, p. 101177, Jan. 2023, ISSN: 25901184. DOI: [10.1016/j.col.2022.101177](https://doi.org/10.1016/j.col.2022.101177).
- [27] M. Zakeri-Nasrabadi and S. Parsa, *ADAFEST: A data-driven apparatus for estimating software testability*, version 1.0.0, Jun. [Accessed: 2023-01-18]. DOI: <https://doi.org/10.5281/zenodo.6683950>. [Online]. Available: <https://m-zakeri.github.io/ADAFEST> (visited on 01/18/2023).
- [28] M. Zakeri-Nasrabadi and S. Parsa, *SENSA: Software entity name suggestion apparatus*, version 1.0.0, Nov. [Accessed: 2023-01-18]. [Online]. Available: <https://github.com/m-zakeri/SENSA> (visited on 01/18/2023).
- [29] S. Hashemifar, S. Parsa, and M. Zakeri, *Mitigating backdoors within deep neural networks in data-limited configuration*, 2023. arXiv: [2311.07417](https://arxiv.org/abs/2311.07417) [cs.LG]. [Online]. Available: <https://arxiv.org/abs/2311.07417>.
- [30] R. Rezvani-Jalal, M. Zakeri, S. Parsa, and A. Hasan-Zarei, "Enhancing malware detection reliability in non-executable files using confidence score prediction," *Int. J. Inf. Secur.*, vol. 25, no. 5, Aug. 2026, ISSN: 1615-5262. DOI: [10.1007/s10207-026-01312-3](https://doi.org/10.1007/s10207-026-01312-3). [Online]. Available: <https://doi.org/10.1007/s10207-026-01312-3>.